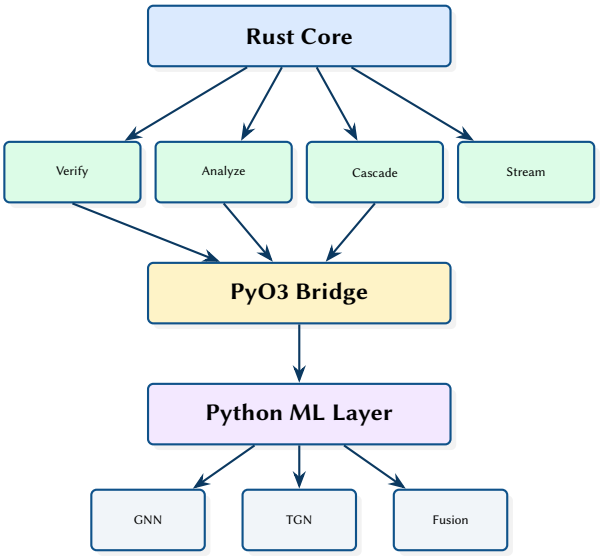
NetAssure: A GNN-Based Real-Time Network Analytics Platform

Formal Verification, Cascade Simulation, and Anomaly Detection

Simon Knight

March 2026 • Version 0.1.0

Last updated: March 11, 2026



Abstract. This report presents NetAssure, a hybrid Rust/Python network analytics platform that provides six complementary analysis paradigms: formal verification via Header Space Analysis, graph-theoretic metrics, Monte Carlo failure cascade simulation, GNN-based anomaly detection, real-time streaming with WebSocket alerting, and topology optimisation. The Rust core (`netassure-core`, `netassure-verify`, `netassure-analyze`, `netassure-cascade`) handles performance-critical deterministic algorithms, while a Python layer provides PyTorch-based machine learning including temporal graph networks and multi-modal fusion scoring. PyO3 bindings bridge the two worlds, exposing verification, analysis, and cascade simulation to the Python ML pipeline.

Contents

1 Introduction and Motivation

Network operators need multiple analysis perspectives to understand complex topologies: formal verification alone cannot predict cascade failures, while machine learning alone cannot guarantee reachability invariants. NetAssure integrates six complementary paradigms into a single platform with a unified topology model.

The six paradigms are:

1. **Formal Verification** — Header Space Analysis (HSA), reachability proofs, loop detection, and traffic isolation checking.
2. **Graph Algorithms** — Centrality, community detection, path diversity, and robustness metrics.
3. **Failure Cascade Modelling** — Monte Carlo simulation with load redistribution and scenario comparison.
4. **Machine Learning (ML)/Graph Neural Network (GNN)** — Anomaly detection, temporal graph networks, and feature attribution via GNNExplainer.
5. **Real-Time Streaming** — Inference pipeline, anomaly detector, alert engine, and WebSocket streaming.
6. **Optimisation** — Critical node identification and redundancy recommendations.

: Hybrid Rust/Python Architecture

Deterministic algorithms (verification, graph metrics, cascade simulation) are implemented in Rust for performance and correctness. Machine learning workloads run in Python (PyTorch/PyTorch Geometric) where the ecosystem is mature. PyO3 bridges the two at the function call level, avoiding serialisation overhead.

1.1 Related Work and Positioning

NetAssure is positioned at the intersection of (i) network configuration verification and policy analysis, (ii) resilience modelling via cascade simulation, and (iii) ML-based anomaly detection on graph-structured telemetry.

Verification and policy analysis. Header Space Analysis (HSA) provides the basis for scalable reachability and isolation checking by representing forwarding behaviour as transformations over header subspaces [0]. Tools such as VeriFlow focus on real-time invariant checking by intercepting control plane updates and validating changes before installation [0]. Batfish models routing and forwarding to answer reachability and security questions over configuration snapshots and what-if scenarios [0]. NetAssure adopts the HSA-style view for formal verification, but explicitly couples it with graph analytics and operational resilience modelling, so operators can explain not only *whether* traffic can flow, but also *how robust* that flow is under failures.

Network programming and semantics. NetKAT provides a rigorous semantic foundation for reasoning about packet-processing functions and composing policies [0]. While NetAssure is not a network programming language, it borrows the discipline of explicit semantics: verification and analysis operate over a single topology model and a consistent set of invariants.

Resilience and cascading failures. Cascading failure models are widely used to study how local faults can amplify into systemic outages in interdependent systems. Motter and Lai show how load redistribution can lead to abrupt breakdowns under targeted perturbations [0]. NetAssure instantiates this idea at the topology level with a round-based simulator, enabling Monte Carlo estimation of cascade depth and scope and explicit before/after comparisons when proposing redundancy.

Graph ML for anomaly detection. GNNs such as GCNs [0], GraphSAGE [0], and GAT [0] support learning from relational structure, while Temporal Graph Networks extend this capability to dynamic event streams [0]. For operator trust, explanation techniques such as GNNExplainer can attribute anomalies to influential subgraphs and features [0]. NetAssure integrates these models into an online pipeline, but keeps probabilistic outputs behind a trust boundary (Section 2) and uses deterministic engines for assertions and guardrails.

: Principled Hybrid: Proofs + Predictions

NetAssure treats verification results as hard constraints and ML results as ranked hypotheses. When the two disagree, the system surfaces the disagreement explicitly rather than collapsing it into a single score.

1.2 Comparison to Other Tools

Tables 1 and 2 summarise how NetAssure compares to common open-source, academic, and commercial tooling. The intent is not to declare a universal winner, but to clarify trade-offs and where NetAssure provides an end-to-end workflow that spans configuration semantics, topology reasoning, resilience modelling, and online inference.

Table 1. Comparison: verification and analysis tools.

Tool	Primary role	Strengths	Weaknesses	Typical usage
NetAssure	Unified topology analysis	Combines HSA, graph metrics, cascade simulation, and ML scoring behind a single topology model	Young project; depends on good topology+telemetry ingestion	Ops: correctness + resilience + RCA
Batfish [0]	Config Q&A	Strong semantics for routing/ACL questions over snapshots	Not streaming-first; less focus on resilience modelling	Pre-change validation / what-if Q&A
VeriFlow [0]	Inline invariants	Real-time checks on control-plane updates	Narrower scope than full analytics stack	Guardrail on updates
NetKAT [0]	Policy semantics	Rigorous foundations for composition/equivalence	Not an operational runtime	Research / policy compilation
Minesweeper [0]	BGP verification	Scales to interdomain policy/config correctness	Focused on BGP, not internal telemetry analytics	ISP BGP safety checks
ARC / Delta-net (placeholder) [0]	Verification frameworks	Unified/incremental verification under change	Placeholder citations here	Continuous verification workflows

Table 2. Comparison: inventory, automation, monitoring, and observability stack.

Tool	Primary role	What it provides	How NetAssure fits
NetBox [0]	Source of truth	Inventory + relationships + intent metadata	Authoritative input for topology ingestion
NAPALM [0] / Netmiko [0]	Device access	Multi-vendor state/config collection	Feed snapshots + confirm post-change state
Ansible [0]	Automation	Desired state workflows	Validate diffs before/after rollout
Prometheus [0]	Metrics TSDB	Time-series metrics + alert rules	Enrich alerts with path impact + suspects
Grafana [0]	Visualisation	Dashboards + alert routing	Surface topology-aware panels/annotations
OpenNMS [0]	NMS platform	Events + polling + inventory adjacencies	Add topology reasoning + RCA
Kentik [0]	NPM/traffic	Flow-based traffic views	Add correctness/resilience context
Datadog [0] / New Relic [0]	Observability	Logs/metrics/traces + built-in ML	Use as telemetry source; add topology semantics
RPKI [0] + BGPStream [0] (RouteViews/RIS) [0]	Routing security/visibility	Origin validation + global route feeds	Map events to internal reachability/blast radius

1.2.1 How NetAssure Complements the Stack

In practice, operators rarely replace their existing tooling; they add new analysis where the current stack has blind spots.

NetBox / automation. NetAssure can consume inventory and intent from a source-of-truth system (e.g. NetBox) and validate changes proposed by automation (NAPALM/Ansible) before and after rollout.

Monitoring. Prometheus/Grafana/OpenNMS (and commercial observability) answer "what happened" at metric scale. NetAssure adds topology-aware root cause ranking, invariant checking, and resilience impact analysis so alerts can be tied to affected paths and critical components.

Forwarding- and control-plane verification. Dedicated verifiers span both intra- and inter-domain settings, ranging from inline invariant checking to incremental verification under continuous changes. Minesweeper targets correctness of BGP configuration and policy at scale [0]. ARC-style frameworks (placeholder citation) and incremental approaches (e.g. Delta-net) emphasise maintaining invariants efficiently as networks evolve (placeholder citations) [0]. NetAssure is positioned as an internal topology analytics platform; when the

scope includes BGP config verification, NetAssure complements (rather than replaces) these specialised verifiers.

Routing security / monitoring. RPKI validation and BGP measurement frameworks (e.g. BGPStream with RouteViews/RIPE RIS feeds) focus on origin/ROA validity, hijack visibility, and macro-level routing events [0]. NetAssure focuses on how those control-plane events map onto internal reachability, affected paths, and local remediation actions.

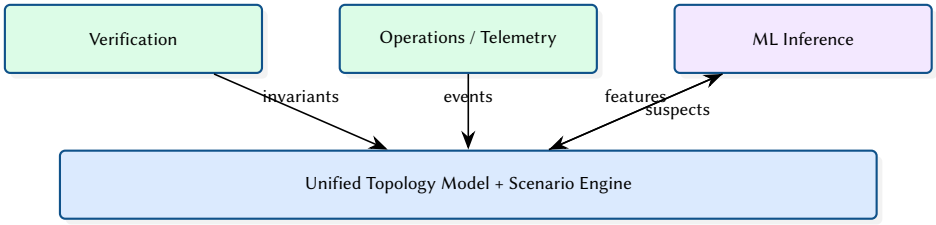


Figure 1. NetAssure's differentiator is the coupling of proofs, telemetry, and predictions through a single scenario engine.

2 System Architecture

2.1 End-to-End Dataflow

Figure 2 shows how topology data and telemetry flow through the system and where deterministic engines and ML inference sit.

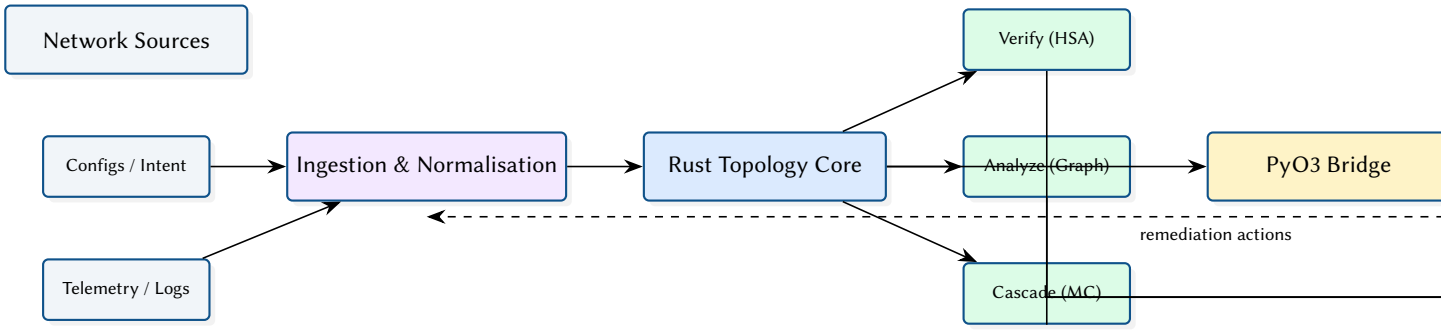


Figure 2. End-to-end dataflow and execution boundaries across Rust and Python.

2.2 Crate Structure

The Rust workspace contains four library crates and a CLI binary:

Table 3. NetAssure Rust crate organisation.

Crate	Role	Key Dependencies
`netassure-core`	Topology model, ingestion	petgraph, serde
`netassure-verify`	HSA, reachability	bit-set, petgraph
`netassure-analyze`	Graph algorithms	rustworkx-core
`netassure-cascade`	Monte Carlo simulation	rayon, rand
`netassure-py`	PyO3 bindings	pyo3
`cli`	Unified CLI	clap, axum, tokio

2.3 Topology Model

The core topology is a `petgraph::StableGraph` with typed node and edge attributes. Stable indices ensure that node/edge removal does not invalidate existing references—critical for cascade simulation where nodes are progressively disabled.

```
pub struct Topology {
    graph: StableGraph<NodeAttr, EdgeAttr>,
    node_index: HashMap<String, NodeIndex>,
}
```

```rust
pub struct NodeAttr {
    pub id: String,
    pub node_type: NodeType,
    pub capacity: f64,
    pub load: f64,
}
```

2.4 Data Model: Entities and Relationships

Figure 3 sketches the core entities and relationships in the topology model used across verification, analytics, and cascade simulation.

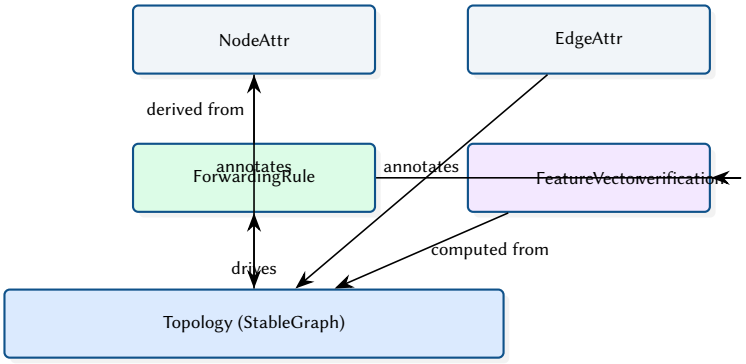


Figure 3. Core topology entities reused by verification, analytics, cascade simulation, and ML feature construction.

2.5 Deployment View

Figure 4 shows a typical deployment split between a Rust service (API + deterministic engines) and a Python inference service, connected through a local boundary.

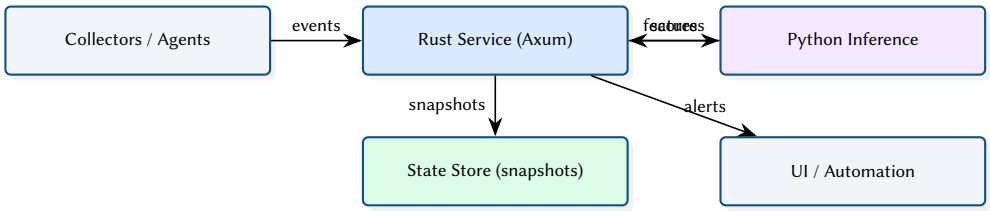


Figure 4. Deployment-oriented view of the Rust API service and Python inference component.

2.6 REST API

The ingestion system exposes an Axum-based HTTP Application Programming Interface (API) for real-time topology synchronisation and prediction queries:

Table 4. REST API endpoints.

Method	Path	Description
GET	`/health`	Liveness probe
GET	`/health/ready`	Readiness (topology synced?)
GET	`/topology`	Full topology JSON
GET	`/predictions`	GNN node predictions
GET	`/anomalies`	Recent anomaly history

2.7 Control Plane vs Data Plane Boundaries

NetAssure keeps a strict separation between what must be correct-by- construction (verification and deterministic analysis) and what is probabilistic (ML inference). Figure 5 sketches the trust boundary used throughout the implementation.

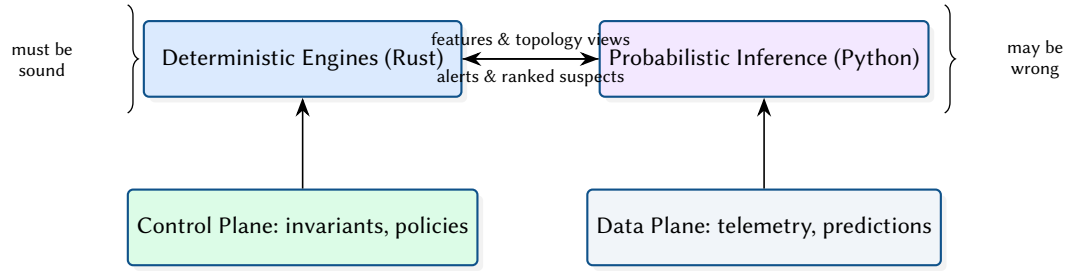


Figure 5. Trust boundary between deterministic analysis and probabilistic inference.

3 Formal Verification

3.1 From Config to Forwarding Semantics

The verifier consumes configuration artefacts (ACLs, routing policy, FIBs) and compiles them into a forwarding semantics used by analysis. Rather than treating the network as a black box, NetAssure makes the intermediate representations explicit: header predicates, actions, and composition. Figure 6 illustrates the concept.

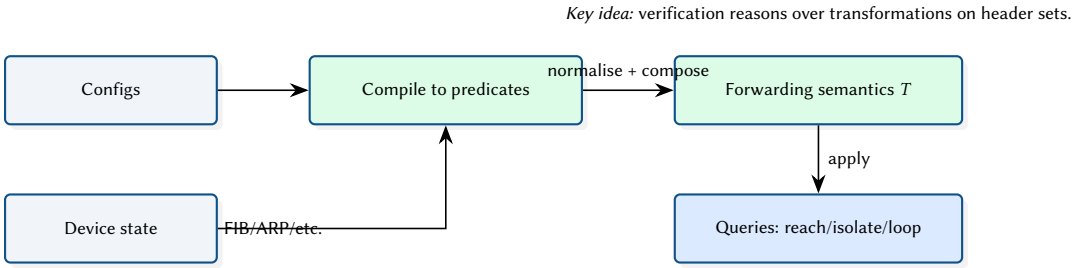


Figure 6. Conceptual compilation pipeline from configuration artefacts to forwarding semantics.

3.2 Header Space Propagation Model

Figure 7 provides a conceptual diagram of header-space propagation across forwarding elements. This abstraction is used to derive reachability, isolation violations, and loop candidates.

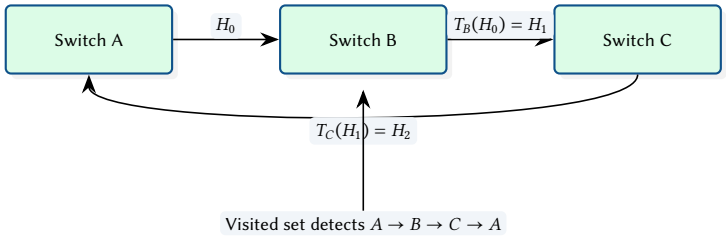


Figure 7. Conceptual header-space propagation and loop detection.

3.3 Fixpoint View: Termination and Over-Approximation

Operationally, reachability queries are solved by repeated propagation until a fixpoint is reached: no new (node,port,header-set) states are discovered. To guarantee termination, implementations use canonical representations (e.g. BDDs) and may apply widening/merging that produces safe over-approximations. Figure 8 illustrates the concept.

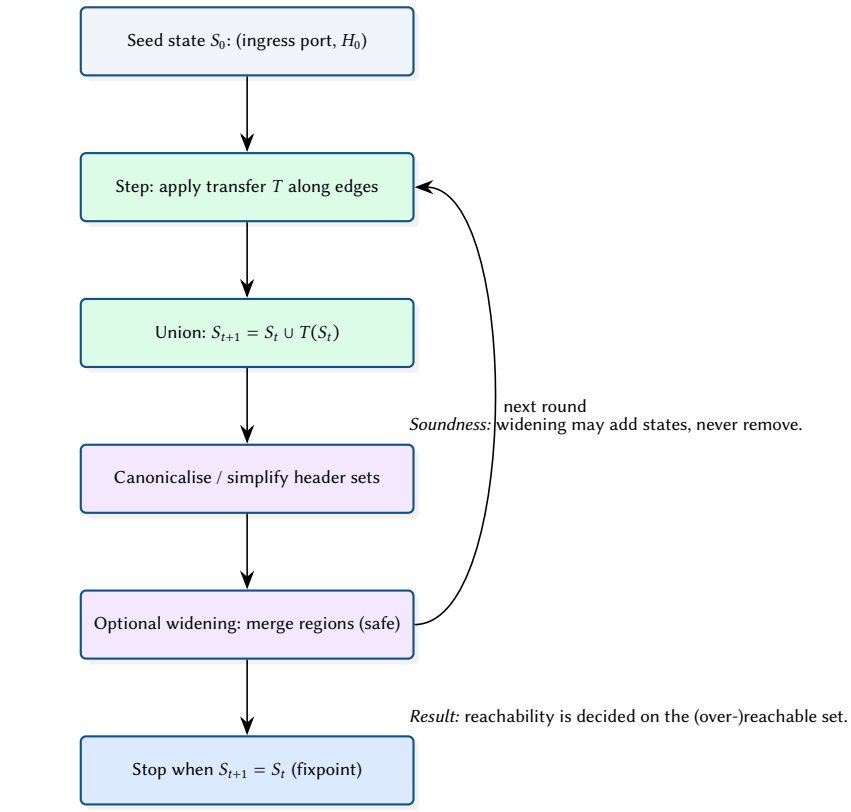


Figure 8. Fixpoint interpretation of header-space propagation with canonicalisation and safe widening.

3.4 Header Space as Geometry (Concept Diagram)

Header space can be visualised as a high-dimensional cube. Predicates carve that space into regions, and forwarding actions map regions from one interface to another. Figure 9 illustrates this idea with a 2D slice (for intuition).

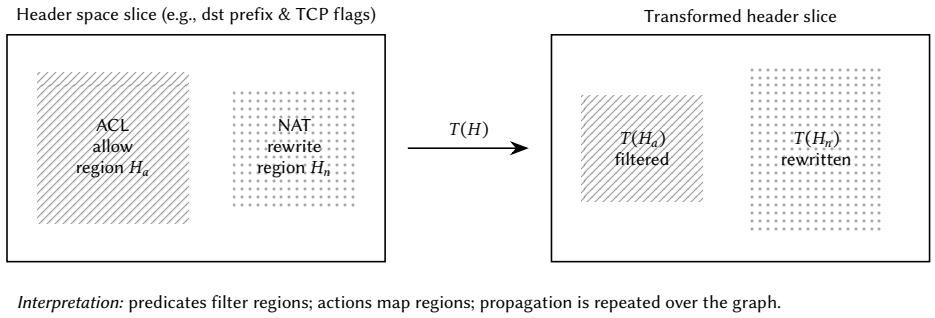
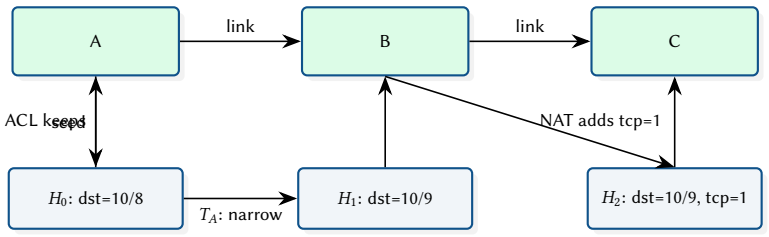


Figure 9. HSA intuition: header predicates carve space into regions; forwarding maps regions across interfaces.

3.5 Worked Reachability Example (Region Propagation)

The following toy example shows how a reachability query propagates a small set of header regions across hops. Each device applies a filter (e.g. ACL) and a transform (e.g. rewrite), producing new regions. The verifier maintains a set of discovered states and iterates until no new regions are produced.



Interpretation: regions become more specific as constraints accumulate; implementation canonicalises/merges to keep the set finite.

Figure 10. Toy propagation of three header regions across three hops for a reachability query.

3.6 Verification Query Types

Operators typically run a small set of repeatable questions. The verification engine exposes these as query templates (Figure 11).

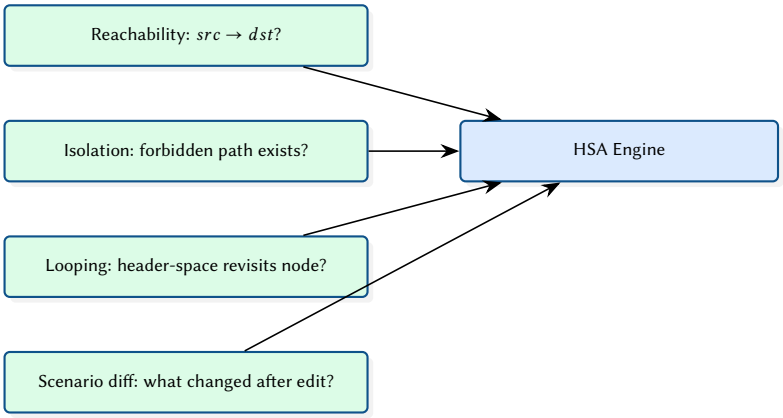


Figure 11. Common verification query templates exposed to operators and automation.

The verification engine implements HSA [0] over the topology graph. Each forwarding rule is modelled as a header-space transfer function; the engine computes reachability by composing transforms along all paths between source and destination.

> **Design Decision 1: Bit-Set Header Representation**
>
> Header spaces are represented as bit-sets rather than ternary vectors. This trades some expressiveness for $O(1)$ intersection and union operations, which dominate the verification runtime. For the network sizes targeted (hundreds of nodes, thousands of rules), this approach is sufficient and avoids the complexity of BDD-based representations.

Loop detection operates by tracking visited nodes during header-space propagation. If a packet's header matches a forwarding rule that would send it to an already-visited node, a loop is reported with the full cycle path.

4 Failure Cascade Simulation

4.1 Cascade Dynamics

Figure 12 illustrates the round-based cascade loop: initial failures remove capacity, load redistributes, and newly overloaded nodes fail until convergence.

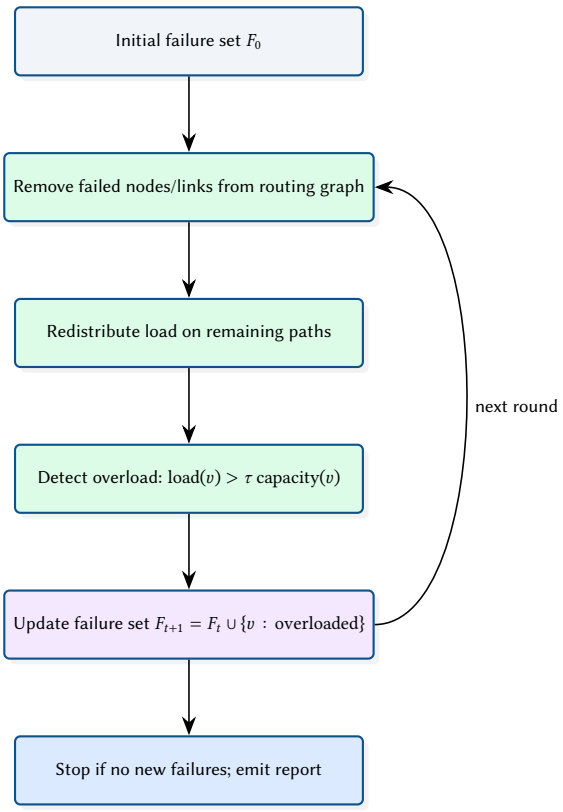


Figure 12. Round-based cascade progression used in the Monte Carlo simulator.

4.2 Toy Example: Rerouting Amplifies Load

To make the cascade mechanics tangible, Figure 13 shows a small topology where an initial link failure forces rerouting onto a longer path. The redistributed load may exceed capacity on a previously non-critical node, creating a secondary failure.

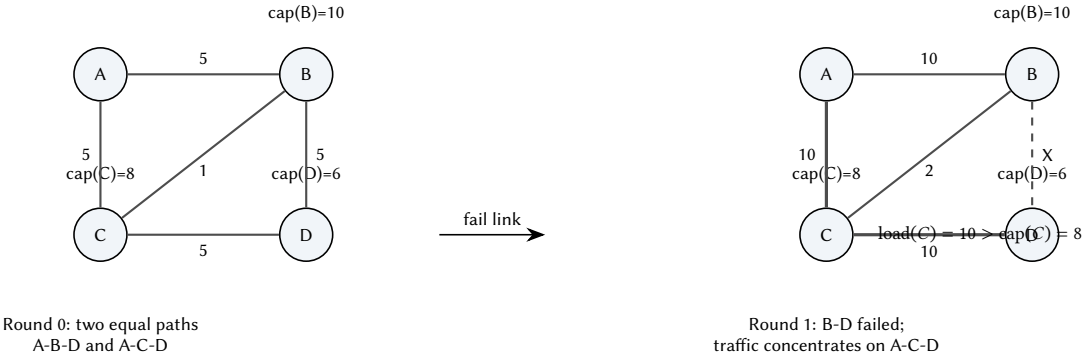


Figure 13. Toy cascade: rerouting after a single failure can overload a different component and trigger secondary failures.

4.3 Load Redistribution as a Potential Function

The cascade loop can be interpreted as a sequence of constrained re-routing steps. A helpful mental model is that each round reduces the feasible routing set, increasing the "stress" on remaining components. Figure 14 visualises this as a potential function over rounds.

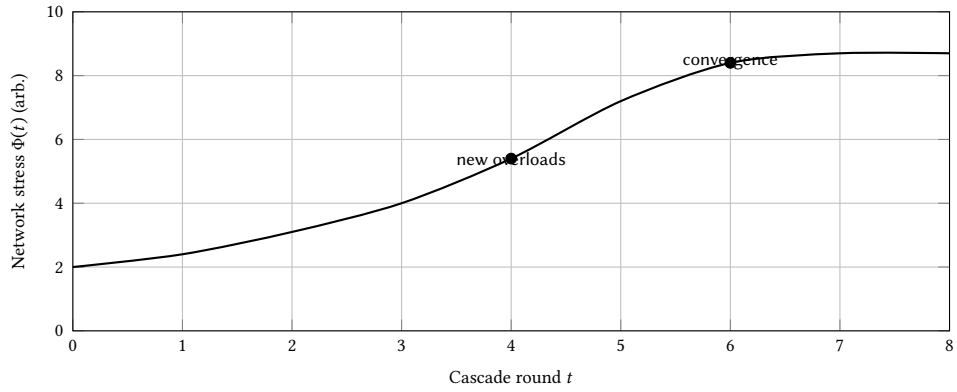


Figure 14. Conceptual view: as failures remove options, stress increases until the cascade converges.

4.4 Scenario Comparison Output

When comparing two topologies (e.g. baseline vs. redundancy upgrade), NetAssure emits paired distributions and deltas. Figure 15 illustrates the intended output.

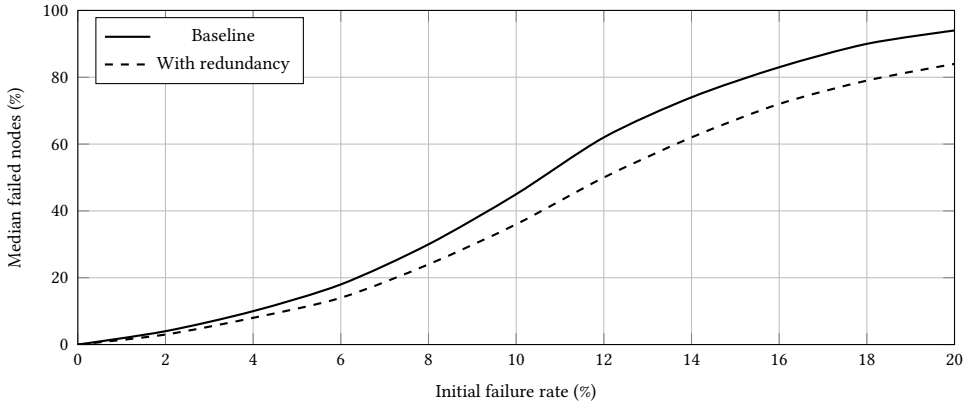
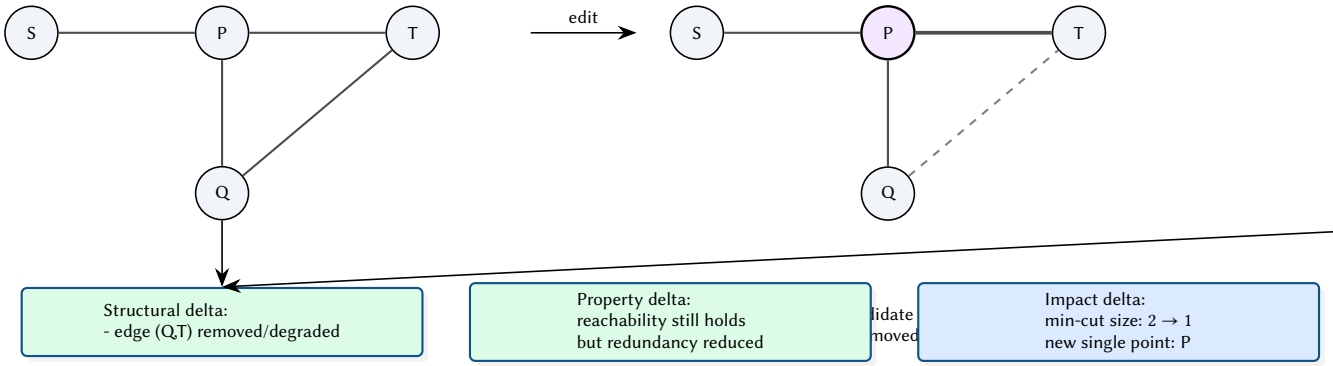


Figure 15. Illustrative scenario comparison: redundancy reduces cascade scope across failure rates.

4.5 Scenario Diff Semantics: What Changed and Why It Matters

A scenario diff compares a baseline topology G and a candidate topology G'. NetAssure computes (i) structural deltas (added/removed nodes/links), (ii) property deltas (reachability/invariant changes), and (iii) impact deltas (affected paths and changes in resilience). The goal is a *minimal explanation* for the observed behavioural change. Figure 16 illustrates this concept.



Minimal explanation: one edge edit collapses path diversity, increasing blast radius.

Figure 16. Scenario diff semantics: structural changes drive property changes, which drive operational impact (paths, cuts, resilience).

4.6 Blast Radius View: Which Flows Lose Redundancy

Operators care about which services and demand pairs are impacted by a change. A structural edit may preserve reachability but reduce the number of disjoint paths for a subset of flows, increasing the blast radius under subsequent failures. Figure 17 shows a toy example where one demand pair loses path diversity after an edit.

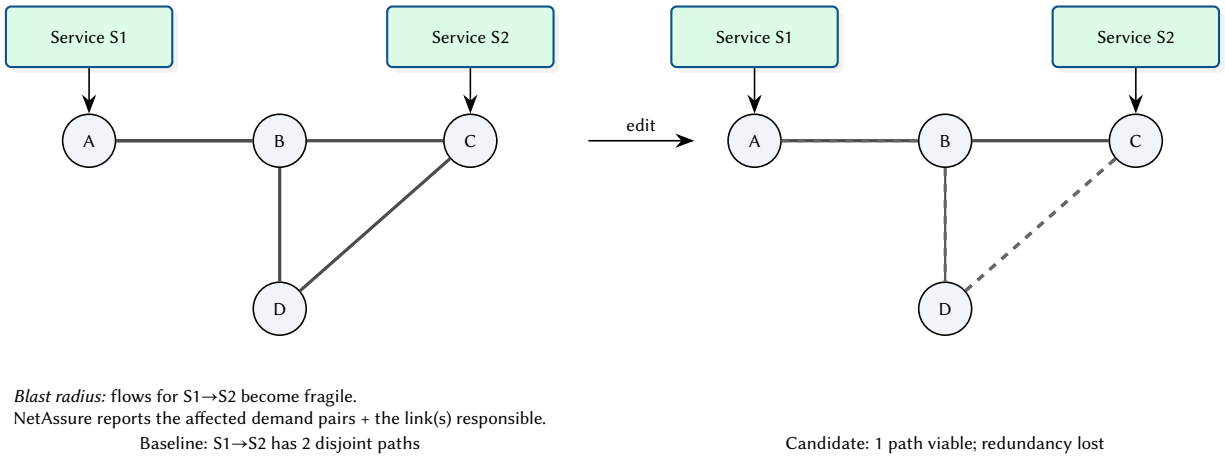


Figure 17. Blast radius concept: a change can reduce path diversity for specific services, increasing risk even if reachability remains.

4.7 Blast Radius Report (Illustrative Output)

Beyond the conceptual view, operators benefit from a compact report that names the most impacted demand pairs/services and the structural causes. Table 5 shows an illustrative format used by NetAssure: for each demand pair, report reachability and an estimate of path diversity before/after the scenario change, along with the critical edge(s) responsible for the drop.

Table 5. Illustrative blast radius report for a scenario diff (synthetic example).

Demand pair	Service	Reachable	k disjoint	Critical edge(s)	Notes
A→C	S1→S2	yes	2 → 1	(D,C)	Redundancy loss; higher risk under any subsequent failure
A→D	S1→S3	yes	2 → 2	–	No diversity change; monitoring only
B→C	S4→S2	yes	3 → 2	(Q,T)	Path diversity reduced; reroute increases load on P
X→Y	S5→S6	no	0 → 0	–	Already unreachable; unchanged by scenario

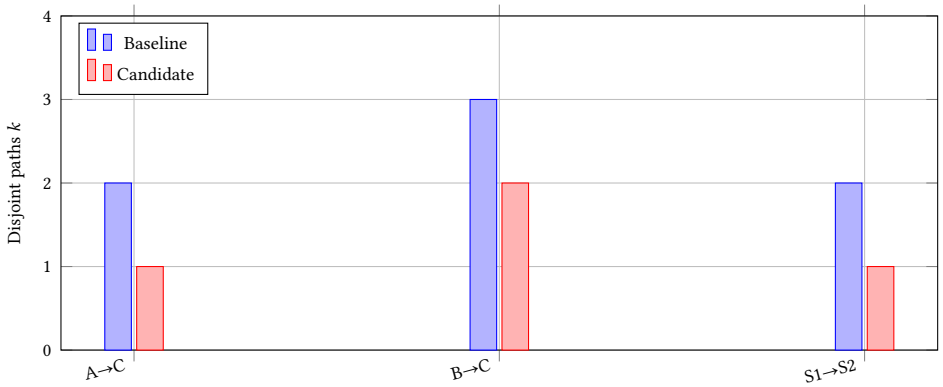


Figure 18. Illustrative summary plot: path diversity k for selected demand pairs before/after a scenario change.

4.8 Example Resilience Curve

The Monte Carlo simulator produces distributions of cascade scope. Figure 19 shows an illustrative curve of median failed fraction vs. initial failure rate.

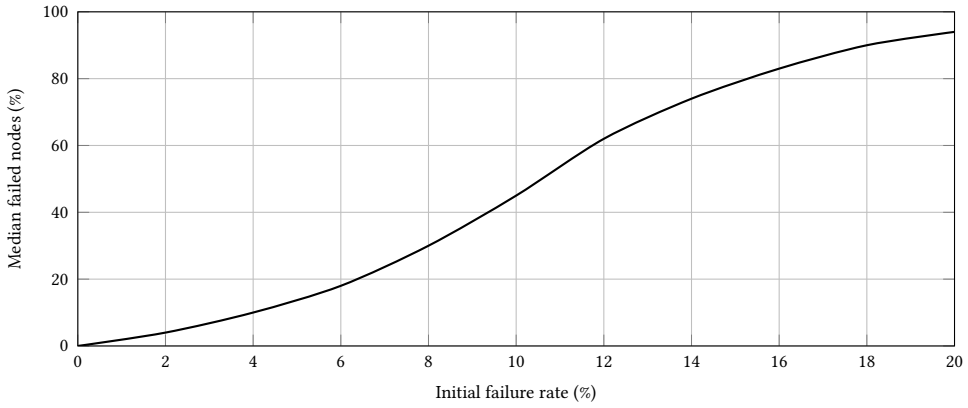


Figure 19. Illustrative resilience curve from Monte Carlo cascade simulation.

The cascade simulator models progressive network failure through load redistribution. When a node or link fails, its traffic is redistributed across remaining paths. If any node exceeds its capacity threshold, it fails in turn, potentially triggering a cascade.

```rust
pub fn simulate_cascade(
    topo: &mut Topology,
    initial_failures: &[NodeIndex],
    redistribution: RedistributionPolicy,
    capacity_threshold: f64,
) -> CascadeReport {
    let mut round = 0;
    let mut failed: HashSet<NodeIndex> = initial_failures.iter().cloned().collect();
    loop {
        let overloaded = find_overloaded(&topo, &failed, capacity_threshold);
        if overloaded.is_empty() { break; }
        failed.extend(overloaded);
        redistribute_load(topo, &failed, &redistribution);
        round += 1;
    }
    CascadeReport { rounds: round, total_failed: failed.len(), .. }
}
```

Monte Carlo mode runs thousands of iterations with random initial failure sets (parameterised by failure percentage), using Rayon for parallel execution. Each iteration produces a cascade depth and scope, enabling statistical analysis of network resilience.

5 Machine Learning Pipeline

5.1 Online Inference and Alerting

Insight:
Scenario comparison mode runs two cascade experiments with different topologies (e.g. before and after adding redundancy) and produces a diff report highlighting improvements in cascade depth, scope, and critical node rankings.

Figure 20 sketches the online loop used by the real-time detector: ingest events, update features, run inference, and push alerts to operators.

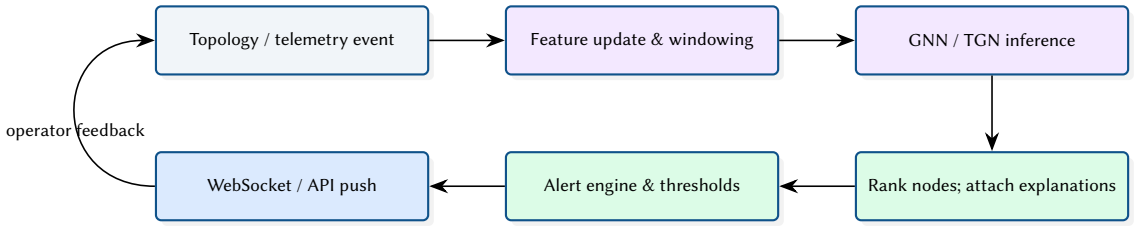


Figure 20. Online inference and alerting pipeline.

5.2 Temporal Message Passing (Concept Diagram)

Temporal models consume events on edges and update node memories over time. Figure 21 illustrates a single event update: messages are computed from source/target state and event features, then aggregated into memory.

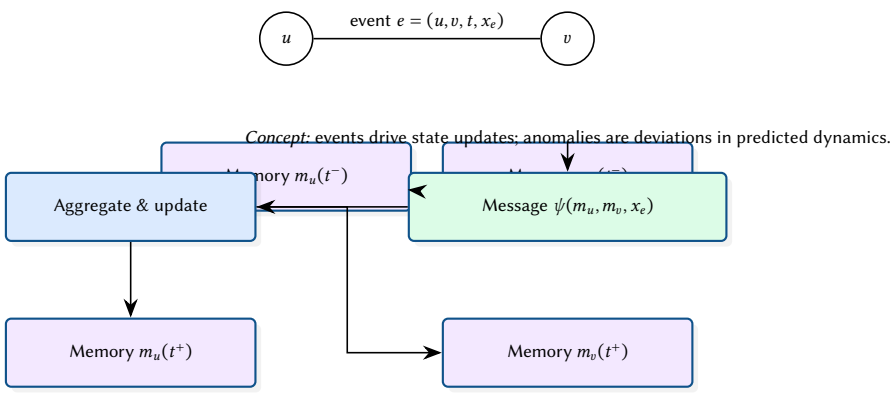


Figure 21. Temporal message passing concept (TGN-like): event features update node memory through messages and aggregation.

5.3 Hybrid Decision: Hard Constraints + Ranked Hypotheses

NetAssure combines deterministic assertions (must-hold invariants) with probabilistic signals (ranked hypotheses). Figure 22 illustrates the decision logic: hard failures trigger immediate violations, while ML scores prioritise investigation among the remaining feasible explanations.

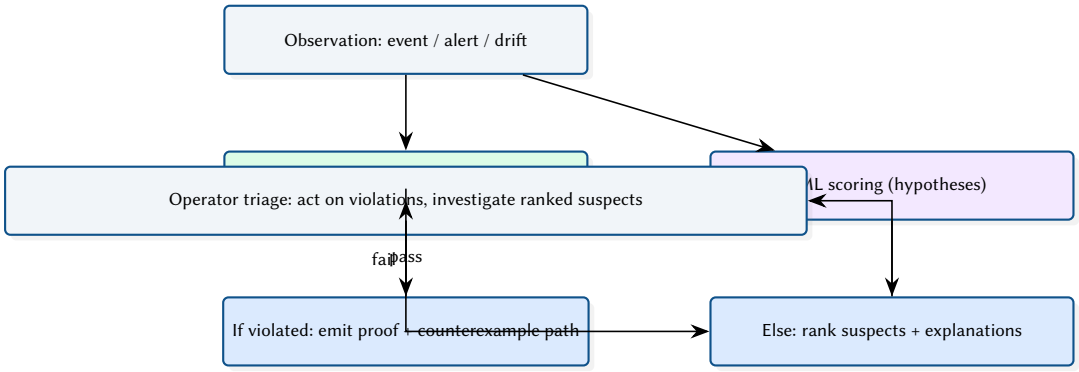


Figure 22. Hybrid decision model: deterministic checks gate correctness; ML prioritises investigation within the feasible space.

5.4 GNN Anomaly Detection

The GNN model [0] operates on a 14-dimensional feature schema per node (degree centrality, betweenness, load ratio, interface counts, etc.) and produces per-node anomaly scores.

5.5 Model Families and Trade-offs

Different model families offer different operational trade-offs. Figure 23 shows an illustrative comparison of latency and operator interpretability (synthetic values).

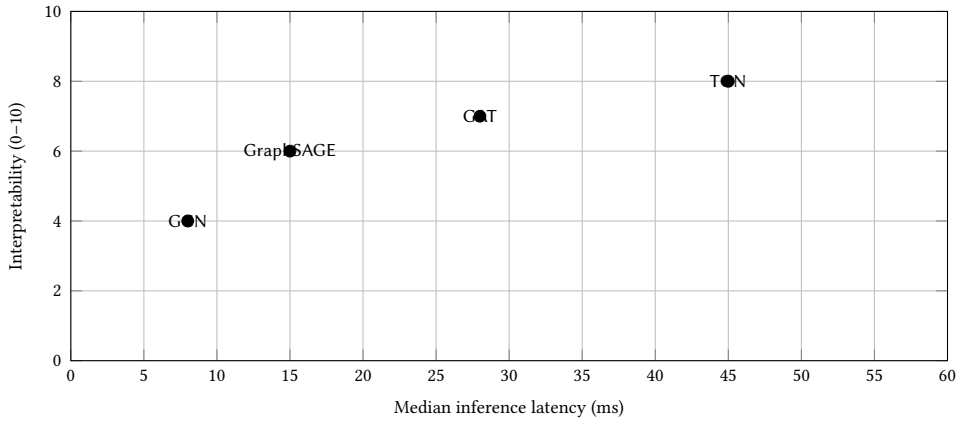
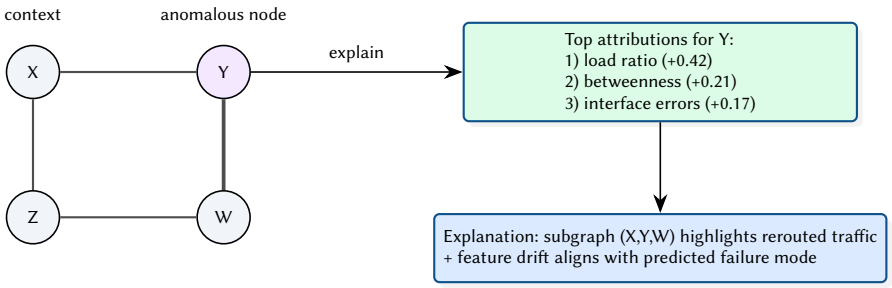


Figure 23. Illustrative trade-off plot: latency vs. interpretability across model families.

5.6 Explanation Output: Subgraph + Feature Attribution (Concept)

To support operator trust, NetAssure presents an anomaly score together with an explanation: a small subgraph and the most influential features. Figure 24 illustrates the intended output format.



Interpretation: show *where* (subgraph) and *why* (features), not just a score.

Figure 24. Conceptual explanation output: anomalous node with a small explanatory subgraph and feature attributions.

5.7 Temporal Graph Networks

The Temporal Graph Network (TGN) model processes time-series topology events (link flaps, load changes) to predict future anomalies. It maintains a per-node memory vector updated at each event, enabling the model to capture temporal patterns such as recurring failures or gradual degradation.

5.8 Multi-Modal Fusion

The fusion scorer combines three signal types:

- **Topology embeddings** (14-dim feature schema)
- **Metrics sequence** (5-dim: latency, loss, jitter, throughput, error rate)
- **Log embeddings** (768-dim from a pre-trained encoder)

> **Design Decision 2: Fusion Architecture**
>
> The fusion model concatenates the three embedding vectors after per-modality projection heads, then passes through a two-layer MLP with dropout. This late-fusion approach allows each modality to be independently pre-trained and evaluated, simplifying debugging when one signal source is noisy or unavailable.

5.9 End-to-End Evaluation Protocol

Figure 25 summarises a practical evaluation protocol for combined deterministic and ML components: validate invariants, then evaluate detection quality and operational cost.

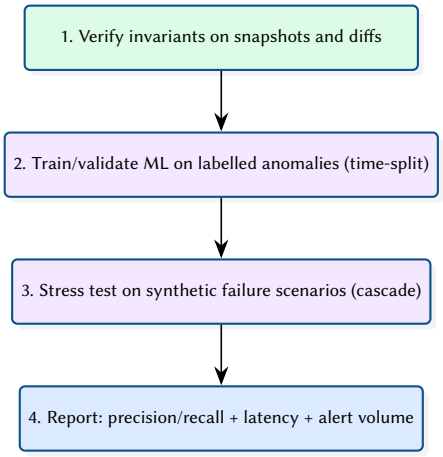


Figure 25. Evaluation protocol across verification, simulation, and ML inference.

6 Design Summary

Table 6. Key design decisions and their rationale.

Decision	Rationale
Hybrid Rust/Python	Deterministic algorithms in Rust for speed; ML in Python for ecosystem maturity
PyO3 bindings	Zero-copy where possible; avoids JSON serialisation on the hot path
StableGraph	Node removal during cascade simulation must not invalidate existing indices
Bit-set HSA	$O(1)$ set operations; sufficient for target network sizes
Late fusion scoring	Independent modality training; graceful degradation when a signal is missing

References

[0] P. Kazemian, G. Varghese, and N. McKeown, "Header space analysis: Static checking for networks," *NSDI*, 2012.

[0] P. Kazemian, R. Govindan, and G. Varghese, "Real time network policy checking using header space analysis," in *NSDI*, 2013.

[0] A. Khurshid, W. Zhou, M. Caesar, and P. B. Godfrey, "VeriFlow: Verifying network-wide invariants in real time," in *NSDI*, 2013.

[0] A. Panda, O. Lahav, K. Lavi, S. Itzhaky, M. Schroeder, and S. Shenker, "Verifying reachability in networks with Batfish," in *NSDI*, 2015.

[0] C. Anderson et al., "NetKAT: Semantic foundations for networks," in *POPL*, 2014.

[0] A. E. Motter and Y.-C. Lai, "Cascade-based attacks on complex networks," *Physical Review E*, 2002.

[0] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *ICLR*, 2017.

[0] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *NeurIPS*, 2017.

[0] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *ICLR*, 2018.

[0] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," *arXiv*, 2020. [Online]. Available: https://arxiv.org/abs/2006.10637

[0] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating explanations for graph neural networks," in *NeurIPS*, 2019.

[0] R. McNutt, A. Panda, M. Budiu, N. McKeown, and S. Shenker, "Minesweeper: An efficient verification tool for BGP configurations," in *NSDI*, 2020.

[0] *ARC: A unified platform for network verification and synthesis*, SIGCOMM, Placeholder citation. Replace with canonical BibTeX (authors/doi/pages) when available., 2016.

[0] *Delta-net: Real-time network verification using network change histories*, SIGCOMM, Placeholder citation. Replace with canonical BibTeX (authors/doi/pages) when available., 2019.

[0] NetBox Project, *NetBox: Infrastructure resource modeling*, 2026. [Online]. Available: https://github.com/netbox-community/netbox

[0] NAPALM Community, *NAPALM: Network automation and programmability abstraction layer with multi-vendor support*, 2026. [Online]. Available: https://github.com/napalm-automation/napalm

[0] K. Byers and contributors, *Netmiko: Multi-vendor SSH library*, 2026. [Online]. Available: https://github.com/ktbyers/netmiko

[0] Red Hat, *Ansible: Automation platform*, 2026. [Online]. Available: https://www.ansible.com

[0] Prometheus Authors, *Prometheus: Monitoring system and time series database*, 2026. [Online]. Available: https://prometheus.io

[0] Grafana Labs, *Grafana: Observability and data visualization*, 2026. [Online]. Available: https://grafana.com/grafana/

[0] OpenNMS Group, *OpenNMS: Network management platform*, 2026. [Online]. Available: https://www.opennms.com

[0] Kentik, *Kentik: Network observability and traffic analytics*, 2026. [Online]. Available: https://www.kentik.com

[0] Datadog, *Datadog: Monitoring and security platform*, 2026. [Online]. Available: https://www.datadoghq.com

[0] New Relic, *New relic: Observability platform*, 2026. [Online]. Available: https://newrelic.com

[0] RIPE NCC and IETF Community, *RPKI: Resource public key infrastructure*, 2026. [Online]. Available: https://rpki.readthedocs.io

[0] CAIDA, *BGPStream: A software framework for live and historical BGP data analysis*, 2026. [Online]. Available: https://bgpstream.caida.org
[0] University of Oregon, *Routeviews project*, 2026. [Online]. Available: https://www.routeviews.org
[0] RIPE NCC, *RIPE RIS: Routing information service*, 2026. [Online]. Available: https://www.ripe.net/analyse/internet-measurements/routing-information-service-ris

List of Figures

List of Tables

List of Design Decisions & Rules

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial architecture report